

**Name:** Charmi Gaddale

**Roll Number:** 23071A0516

**AIM:** Implement Support vector machines learning algorithm- Linear and Non -Linear.

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
from sklearn.datasets import load_breast_cancer

# Split dataset
from sklearn.model_selection import train_test_split

# Feature scaling
from sklearn.preprocessing import StandardScaler

# SVM model
from sklearn.svm import SVC

# Evaluation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# Load Breast Cancer dataset
data = load_breast_cancer()

# Features
X = data.data

# Target labels
y = data.target

# Convert to DataFrame
df = pd.DataFrame(X, columns=data.feature_names)

# Add target column
df["target"] = y

# Display first rows
print(df.head())

# Total records
print("Total records:", df.shape[0])
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0      17.99         10.38         122.80      1001.0         0.11840
1      20.57         17.77         132.90      1326.0         0.08474
2      19.69         21.25         130.00      1203.0         0.10960
3      11.42         20.38          77.58        386.1         0.14250
4      20.29         14.34         135.10      1297.0         0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0         0.27760         0.3001         0.14710         0.2419
1         0.07864         0.0869         0.07017         0.1812
2         0.15990         0.1974         0.12790         0.2069
3         0.28390         0.2414         0.10520         0.2597
4         0.13280         0.1980         0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0         0.07871  ...         17.33         184.60         2019.0
1         0.05667  ...         23.41         158.80         1956.0
2         0.05999  ...         25.53         152.50         1709.0
3         0.09744  ...         26.50          98.87          567.7
4         0.05883  ...         16.67         152.20         1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
```

|   |        |        |        |        |
|---|--------|--------|--------|--------|
| 0 | 0.1622 | 0.6656 | 0.7119 | 0.2654 |
| 1 | 0.1238 | 0.1866 | 0.2416 | 0.1860 |
| 2 | 0.1444 | 0.4245 | 0.4504 | 0.2430 |
| 3 | 0.2098 | 0.8663 | 0.6869 | 0.2575 |
| 4 | 0.1374 | 0.2050 | 0.4000 | 0.1625 |

|   | worst symmetry | worst fractal dimension | target |
|---|----------------|-------------------------|--------|
| 0 | 0.4601         | 0.11890                 | 0      |
| 1 | 0.2750         | 0.08902                 | 0      |
| 2 | 0.3613         | 0.08758                 | 0      |
| 3 | 0.6638         | 0.17300                 | 0      |
| 4 | 0.2364         | 0.07678                 | 0      |

[5 rows x 31 columns]  
Total records: 569

# Split dataset into training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

# Standardize the dataset

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

# Create Linear SVM model

```
linear_svm = SVC(kernel='linear')
```

# Train the model

```
linear_svm.fit(X_train, y_train)
```

▼ SVC ⓘ ?  
SVC(kernel='linear')

# Predict using linear SVM

```
linear_pred = linear_svm.predict(X_test)
```

# Evaluate model

```
print("Linear SVM Accuracy:", accuracy_score(y_test, linear_pred))
```

```
print(classification_report(y_test, linear_pred))
```

```
Linear SVM Accuracy: 0.956140350877193
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.93      | 0.95   | 0.94     | 43      |
| 1            | 0.97      | 0.96   | 0.96     | 71      |
| accuracy     |           |        | 0.96     | 114     |
| macro avg    | 0.95      | 0.96   | 0.95     | 114     |
| weighted avg | 0.96      | 0.96   | 0.96     | 114     |

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
cm_linear = confusion_matrix(y_test, linear_pred)
```

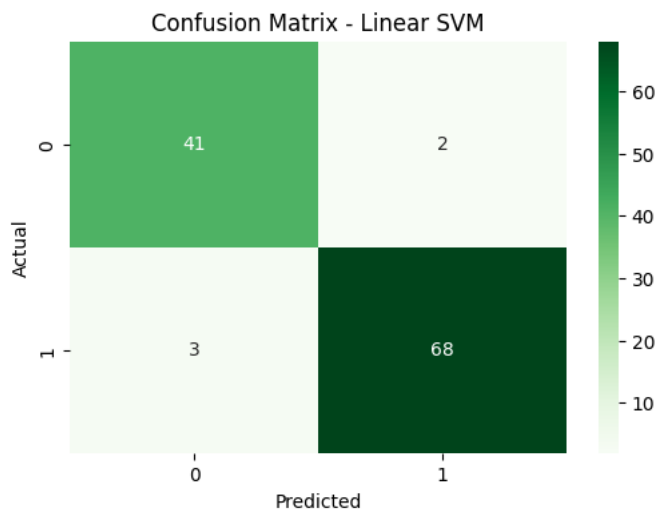
```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm_linear, annot=True, cmap="Greens")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.title("Confusion Matrix - Linear SVM")

plt.show()
```



```
# Create Non-linear SVM model using RBF kernel

nonlinear_svm = SVC(kernel='rbf')

# Train the model
nonlinear_svm.fit(X_train, y_train)
```

▼ SVC ⓘ ?  
SVC()

```
# Predict using nonlinear SVM
nonlinear_pred = nonlinear_svm.predict(X_test)

# Evaluate model
print("Non-Linear SVM Accuracy:", accuracy_score(y_test, nonlinear_pred))

print(classification_report(y_test, nonlinear_pred))
```

```
Non-Linear SVM Accuracy: 0.9824561403508771
precision    recall  f1-score   support

      0       1.00      0.95      0.98         43
      1       0.97      1.00      0.99         71

   accuracy          0.98         114
  macro avg          0.99      0.98      0.98         114
 weighted avg          0.98      0.98      0.98         114
```

```
# Confusion matrix visualization

import seaborn as sns

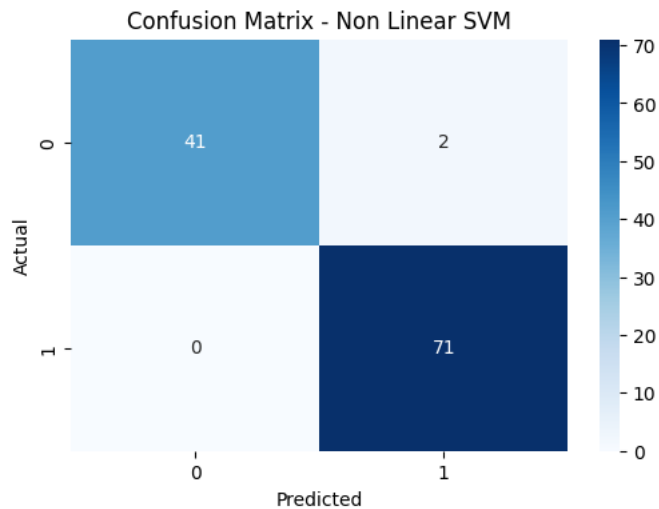
cm = confusion_matrix(y_test, nonlinear_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix - Non Linear SVM")
```

```
plt.show()
```



```
# Example new data prediction
```

```
new_data = [X_test[0]]
print(new_data)
prediction = nonlinear_svm.predict(new_data)
print(prediction)
```

```
if prediction[0] == 0:
    print("Prediction: Malignant Tumor")
else:
    print("Prediction: Benign Tumor")
```

```
[array([-0.46649743, -0.13728933, -0.44421138, -0.48646498,  0.28085007,
        0.04160589, -0.11146496, -0.26486866,  0.41524141,  0.13513744,
        -0.02091509, -0.29323907, -0.17460869, -0.2072995 , -0.01181432,
        -0.35108921, -0.1810535 , -0.24238831, -0.33731758, -0.0842133 ,
        -0.2632354 , -0.14784208, -0.33154752, -0.35109337,  0.48001942,
        -0.09649594, -0.03583041, -0.19435087,  0.17275669,  0.20372995])]
[1]
Prediction: Benign Tumor
```

**Inference:** In this experiment, Support Vector Machine algorithms were implemented using both linear and non-linear kernels on the Breast Cancer dataset. The dataset was preprocessed and scaled before training the models. The linear SVM attempts to separate the data using a straight hyperplane, while the non-linear SVM uses kernel functions such as the radial basis function to capture complex patterns in the data. The models were evaluated using accuracy and classification metrics. The non-linear SVM generally performs better when the data is not linearly separable due to its ability to model complex decision boundaries.